



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECJ3483-01
WEB TECHNOLOGY

Lecturer: DR. NORIZAM BIN KATMON

Group Project Lab Exercise

Name	Matric No
IZZAT FAKHRULLAH BIN KARIM	A23CS0089
RAMI YASSEIN ELTAYEB MOHAMED	A23CS0022
AMMAR ABDULRAHMAN ANAAM MUDHSH	A23CS0287
NOUREDIN MAMDOUH M. E. ELNAMLA	A23CS0020

1.0 Phase 1	3
1.1 Security Investigation	3
2.0 Phase 2	9
2.1 Weakness Classification and Discussion	9
3.0 Phase 3	10
3.1 Problem-Solution Mapping	10
4.0 Phase 4	11
4.1 Before-and-After Testing Evidence	11
4.2 After-Fix Screenshots	12
Test 1: Add BMI with Invalid Data	12
Test 2: Access Another User's BMI Record	13
Test 3: Access Staff Route as Normal User	13
Test 4: Access Admin Route as Normal User	14
Test 5: Submit XSS Payload in Notes	15
Test 6: Check API Response	17
Test 7: Try SQL Injection Input	19
Test 8: Try to Update Protected Fields	21
5.0 Reflections	23

1.0 Phase 1

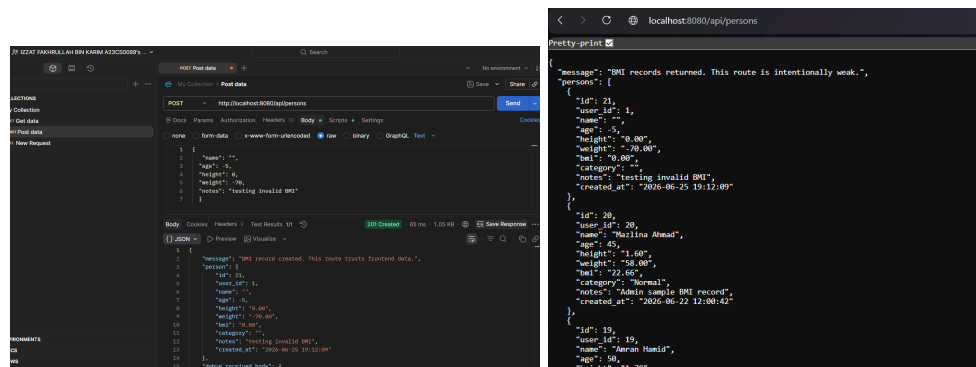
1.1 Security Investigation

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness	Why?
1	Add BMI with invalid data	Backend should reject invalid data	Backend accepts the invalid BMI data if sent through Postman/Thunder Client. Frontend validation can be bypassed.	Yes	Invalid data should not be stored because frontend validation can be bypassed.
2	Access another user's BMI record	Backend should deny access	A normal user can request another user's record using API and the backend returns it.	Yes	Users should only access records that belong to them unless they are staff/admin.
3	Access staff route as normal user	Backend should deny access	Normal user can access /api/staff/persons and view all BMI records.	Yes	Normal users should not access staff monitoring features.
4	Access admin route as normal user	Backend should deny access	Normal user can access /api/admin/users and see all users.	Yes	Admin functions must be restricted to admin role only.
5	Submit XSS payload in notes	Payload should display as text	The notes field is rendered using v-html, so payload	Yes	User-generated content should not run as JavaScript in the

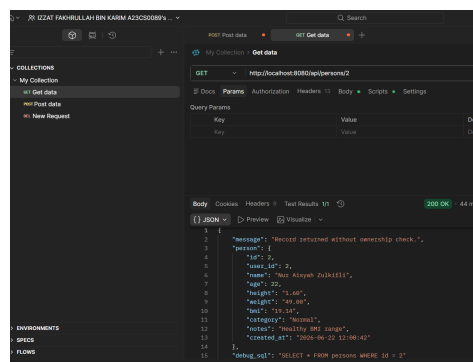
			like can execute in the browser.		browser.
6	Check API response	Password/hash should not be visible	API responses such as /api/admin/users expose password and password_hash	Yes	API should return only necessary data and hide sensitive fields.
7	Try SQL Injection input	Input should be treated as data	Login query uses string concatenation, so SQL injection input may affect the query behavior.	Yes	SQL Injection can happen if input is directly joined into SQL queries.
8	Try to update protected fields	Backend should reject or ignore protected fields	Backend accepts update payloads containing protected fields like user_id, bmi, and category.	Yes	Users should not control fields such as user_id, role, bmi, or category.

No/Title	Screenshots
----------	-------------

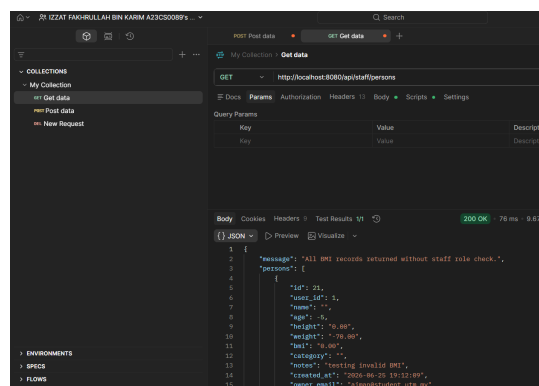
1-Add BMI with invalid data



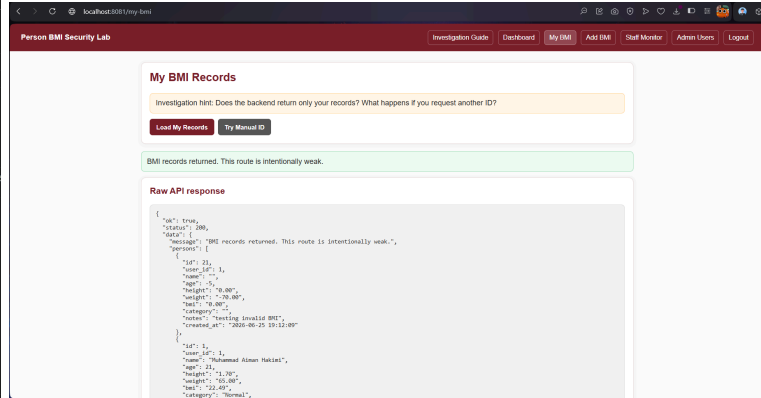
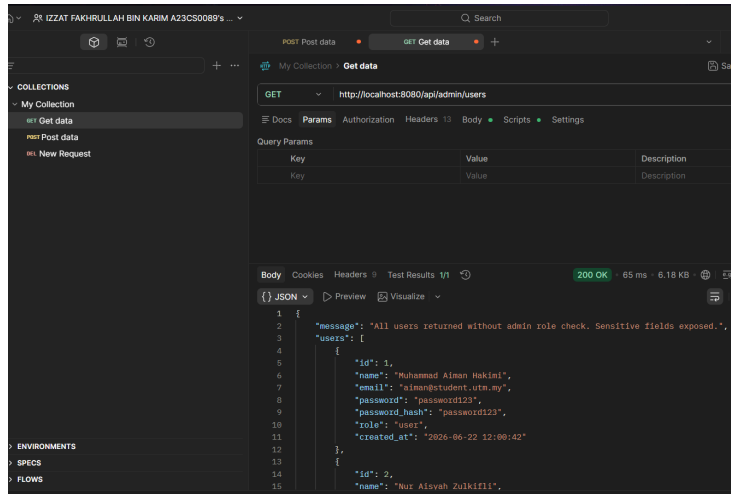
2-Access another user's BMI record



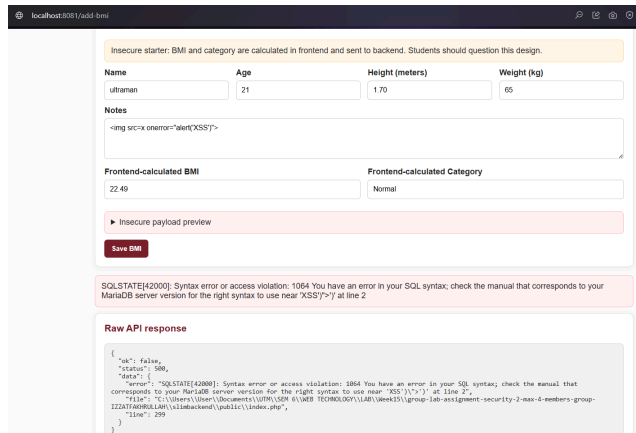
3-Access staff route as normal user



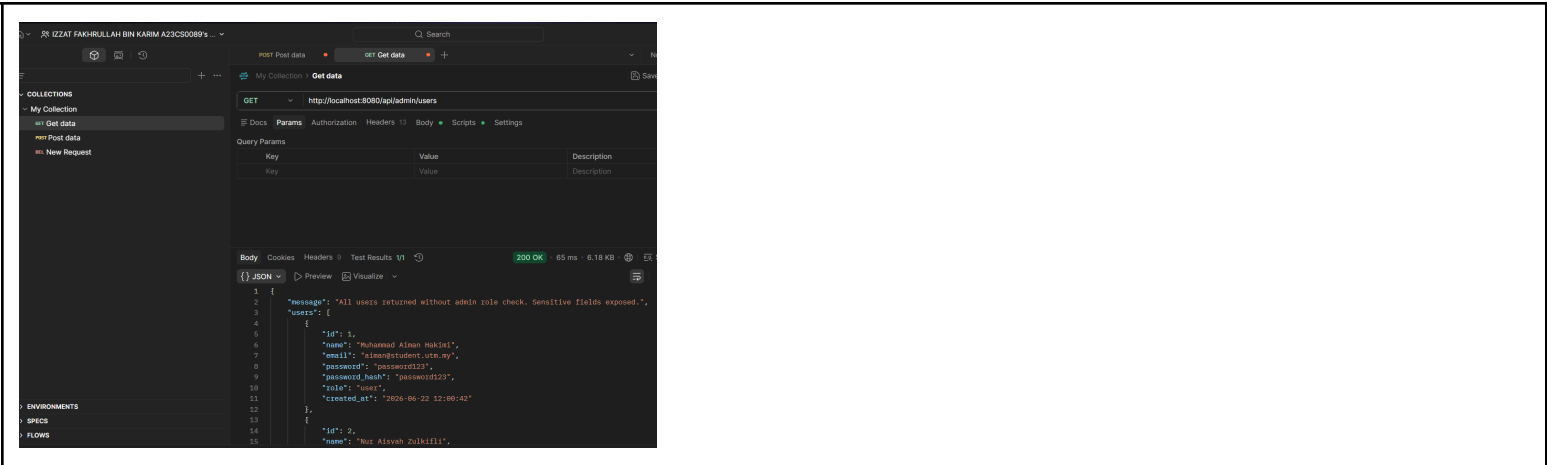
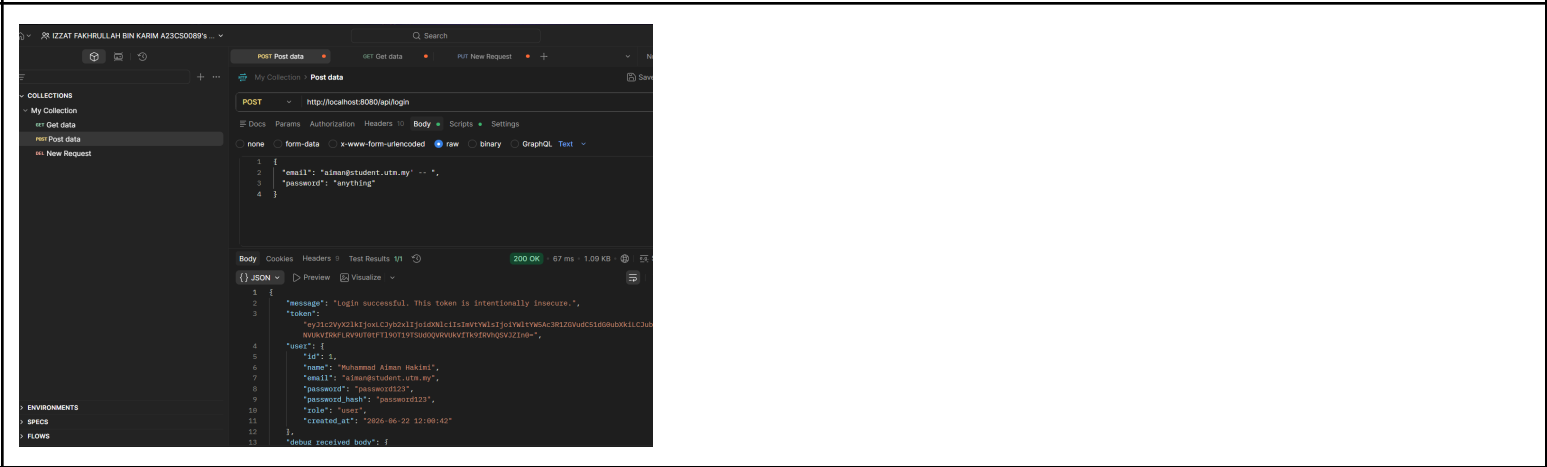
4-Access admin route as normal user



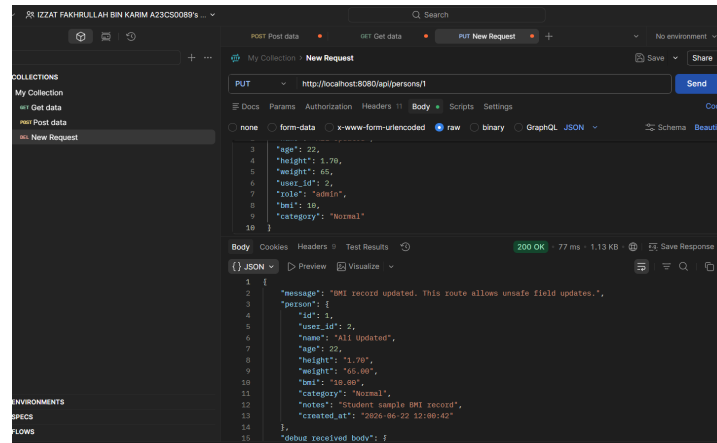
5-XSS payload in notes



6-API response exposes sensitive data

7-SQL
Injection
input

8-Update protected fields



The system contains several backend and frontend security weaknesses. The main issue is that the backend trusts user-controlled input and does not fully enforce validation, authentication, authorization, safe API responses, and safe output rendering.

2.0 Phase 2

2.1 Weakness Classification and Discussion

Weakness Found	Frontend / Backend / API / Database	Security Category	Possible Risk
Invalid BMI data accepted	Backend	Missing backend validation	Invalid BMI data can be stored in the database
User can access another user's BMI record	API / Backend	Broken Object Level Authorization	Privacy violation because users can view records that do not belong to them
Normal user can access staff route	API / Backend	Broken Function Level Authorization	Normal user can view staff-level BMI records
Normal user can access admin route	API / Backend	Broken Function Level Authorization	Unauthorized user may access admin functions
XSS payload executes in notes	Frontend	Cross-Site Scripting	Malicious script can run in the browser
API response exposes password or password_hash	API / Backend	Sensitive Data Exposure	Sensitive user credential information may be exposed
SQL Injection input may affect query	Backend / Database	SQL Injection	Unauthorized access or database query manipulation may happen
User can update protected fields	API / Backend	Mass Assignment	User may change fields such as user_id, role, bmi, or category

3.0 Phase 3

3.1 Problem-Solution Mapping

Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI data accepted	Backend does not validate the input properly	Backend	Validate name, age, height, and weight before saving data	Submit invalid BMI data again using Postman/Thunder Client
User can access another user's BMI record	No ownership check is applied	Backend API	Compare the logged-in user ID from JWT with the record owner ID	Try to access another user's BMI record again
Normal user can access staff route	No role checking for staff route	Backend API	Allow only staff and admin roles to access staff routes	Send request to staff route using normal user token
Normal user can access admin route	No role checking for admin route	Backend API	Allow only admin role to access admin routes	Send request to admin route using normal user token
XSS payload executes in notes	Frontend renders user input as HTML	Frontend	Display user-generated content as text instead of HTML	Submit XSS payload again and check whether alert executes
API response exposes password or password_hash	API returns sensitive user fields	Backend API	Return only necessary fields such as id, name, email, and role	Check API response again and confirm password fields are removed
SQL Injection input may affect query	SQL query uses string concatenation	Backend / Database	Use prepared statements for database queries	Try SQL Injection input again in login or search field
User can update protected fields	Backend accepts fields directly from request body	Backend API	Allow only safe fields such as name, age, height, weight, and notes	Send update request with user_id, role, bmi, or category again

4.0 Phase 4

4.1 Before-and-After Testing Evidence

Test Case	Before Fix	Fix Implemented	After Fix
Add BMI with invalid data	Backend accepted invalid BMI data from Postman/Thunder Client	Added backend validation for name, age, height, and weight	Backend rejects invalid data with error response
Access another user's BMI record	A normal user could access another user's BMI record by changing the record ID in the URL.	Added owner-based access control by checking logged-in user ID with record owner ID	Backend returns access denied when user accesses another user's record
Access staff route as normal user	Normal user could access /api/staff/persons	Added role-based access control for staff route	Backend allows only staff or admin to access staff route
Access admin route as normal user	Normal user could access /api/admin/users	Added role-based access control for admin route	Backend allows only admin to access admin route
Submit XSS payload in notes	XSS payload could execute because notes were rendered as HTML	Removed unsafe HTML rendering and displayed notes as plain text	XSS payload is displayed as text and does not execute
Check API response	API response exposed sensitive fields such as password or password_hash	Removed sensitive fields from API response	API returns only necessary fields such as id, name, email, and role
Try SQL Injection input	SQL Injection input could affect query behavior	Replaced string SQL query with prepared statements	SQL Injection input is treated as normal data

Try to update protected fields	Backend accepted protected fields such as user_id, role, bmi, or category	Allowed only safe fields to be updated and controlled protected fields in backend	Protected fields are rejected or ignored by backend
--------------------------------	---	---	---

4.2 After-Fix Screenshots

Test 1: Add BMI with Invalid Data

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/api/persons
- Body Type:** raw
- Request Body:**

```

1  {
2    "name": "",
3    "age": -5,
4    "height": 0,
5    "weight": -70,
6    "notes": "testing invalid BMI"
7  }

```
- Response:** 400 Bad Request (50 ms, 655 B)
- Response Body (JSON):**

```

1  {
2    "error": "Invalid BMI data",
3    "details": {
4      "name": "Name is required",
5      "age": "Age must be between 1 and 120",
6      "height": "Height must be between 0.5 and 2.5 meters",
7      "weight": "Weight must be between 2 and 300 kg"
8    }
9  }

```

Result: Backend rejected invalid BMI data.

Test 2: Access Another User's BMI Record

GET ▼ <http://localhost:8080/api/persons/2>

Docs Params Authorization **Headers 13** Body ● Scripts ● Settings

Headers 👁 9 hidden

Key	Value	Description
✓ Content-Type	application/json	
✓ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...	
✓		
✓		
Key	Value	Description

Body Cookies Headers 9 Test Results 0/1 🔄 **403 Forbidden** • 58 ms • 4

{ } JSON ▼ ▶ Preview | ⚙ Try with different auth credentials ▼

```
1 {
2   "error": "Access denied"
3 }
```

Result: Backend denied access to another user's record.

Test 3: Access Staff Route as Normal User

GET http://localhost:8080/api/staff/persons

Docs Params Authorization Headers 13 Body Scripts Settings

Headers 9 hidden

Key	Value	Description
✓ Content-Type	application/json	
✓ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...	
✓		
✓		
Key	Value	Description

Body Cookies Headers 9 Test Results 0/1 403 Forbidden 72 ms

{ } JSON Preview Try with different auth credentials

```
1 {
2   "error": "Staff access required"
3 }
```

Result: Backend blocked normal user from staff route.

Test 4: Access Admin Route as Normal User

GET ▼ http://localhost:8080/api/admin/users

Docs Params Authorization **Headers 13** Body ● Scripts ● Settings

Headers 👁 9 hidden

Key	Value	Description
✓ Content-Type	application/json	
✓ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...	
✓		
✓		
Key	Value	Description

Body Cookies Headers 9 Test Results 0/1 🕒 **403 Forbidden** • 72 ms • 431 B •

{ } JSON ▼ ▶ Preview ⚙ Try with different auth credentials ▼

```
1 {
2   "error": "Admin access required"
3 }
```

Result: Backend blocked normal user from admin route.

Test 5: Submit XSS Payload in Notes

POST

http://localhost:8080/api/persons

Docs

Params

Authorization

Headers 12

Body

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

☐ GraphQL JSON

```
1  {
2    "name": "XSS Test",
3    "age": 21,
4    "height": 1.7,
5    "weight": 65,
6    "notes": "<img src=x onerror=\"alert('XSS')\">"
7  }
```

Body

Cookies

Headers 9

Test Results 1/1

🕒

201 Created

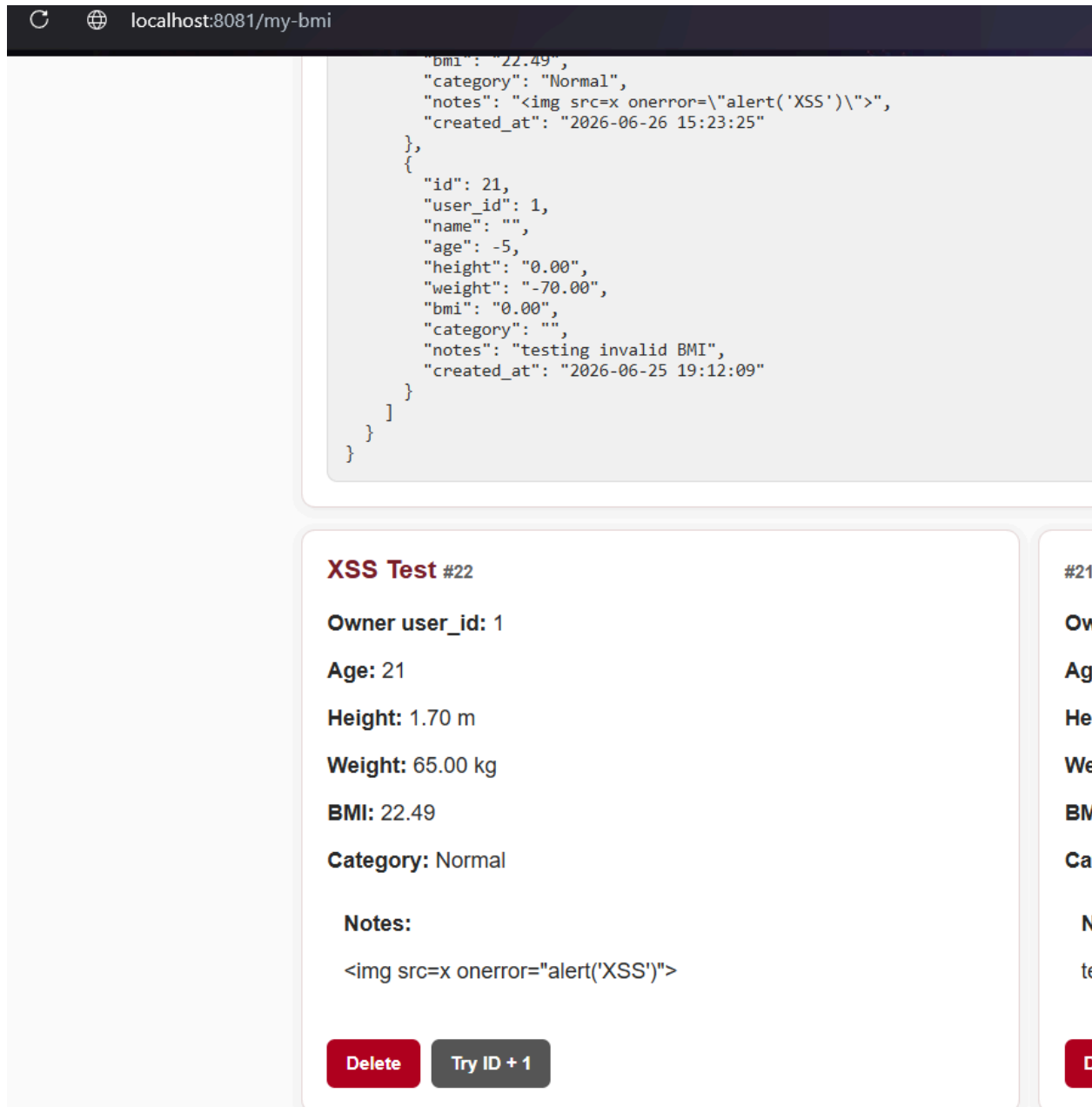
• 67 ms • 78

{ } JSON

▶ Preview

🖼 Visualize

```
1  {
2    "message": "BMI record created. Backend calculated BMI and category.",
3    "person": {
4      "id": 22,
5      "user_id": 1,
6      "name": "XSS Test",
7      "age": 21,
8      "height": "1.70",
9      "weight": "65.00",
10     "bmi": "22.49",
11     "category": "Normal",
12     "notes": "<img src=x onerror=\"alert('XSS')\">",
13     "created_at": "2026-06-26 15:23:25"
14   }
15 }
```



Result: XSS payload was displayed as text and did not execute.

Test 6: Check API Response

POST <http://localhost:8080/api/login>

Docs Params Authorization Headers 12 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema

```
1 {
2   "email": "amran.hamid@utm.my",
3   "password": "password123"
4 }
```

Body Cookies Headers 9 Test Results 1/1 200 OK • 137 ms • 782 B • Save

{ } JSON Preview Visualize

```
1 {
2   "message": "Login successful.",
3   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxOSwicm9sZSI6ImFkbWluIiwiaWF0IjoxNzgyNDU0OTMzLCJleHAiOjE3ODI0NjI1MzN9.5QJtc2VnWA8bfhZF6bJ4riVvcZCxUn2aiIMxSAw0WqI",
4   "user": {
5     "id": 19,
6     "name": "Amran Hamid",
7     "email": "amran.hamid@utm.my",
8     "role": "admin",
9     "created_at": "2026-06-22 12:00:42"
10  }
11 }
```

GET ▼ http://localhost:8080/api/admin/users

Docs Params Authorization **Headers 8** Body Scripts Settings

Headers 7 hidden

Key	Value
✓ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpX...
Key	Value

Body Cookies Headers 9 Test Results ↺

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼

```
2  "message": "All users returned for admin.",
3  "users": [
4    {
5      "id": 1,
6      "name": "Muhammad Aiman Hakimi",
7      "email": "aiman@student.utm.my",
8      "role": "user",
9      "created_at": "2026-06-22 12:00:42"
10   },
11   {
12     "id": 2,
13     "name": "Nur Aisyah Zulkifli",
14     "email": "aisyah@student.utm.my",
15     "role": "user",
16     "created_at": "2026-06-22 12:00:42"
```

Result: Sensitive fields such as password and password_hash were removed.

Test 7: Try SQL Injection Input

POST ▼ http://localhost:8080/api/login

☰ Docs Params Authorization Headers 12 **Body** ● Scripts ● Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

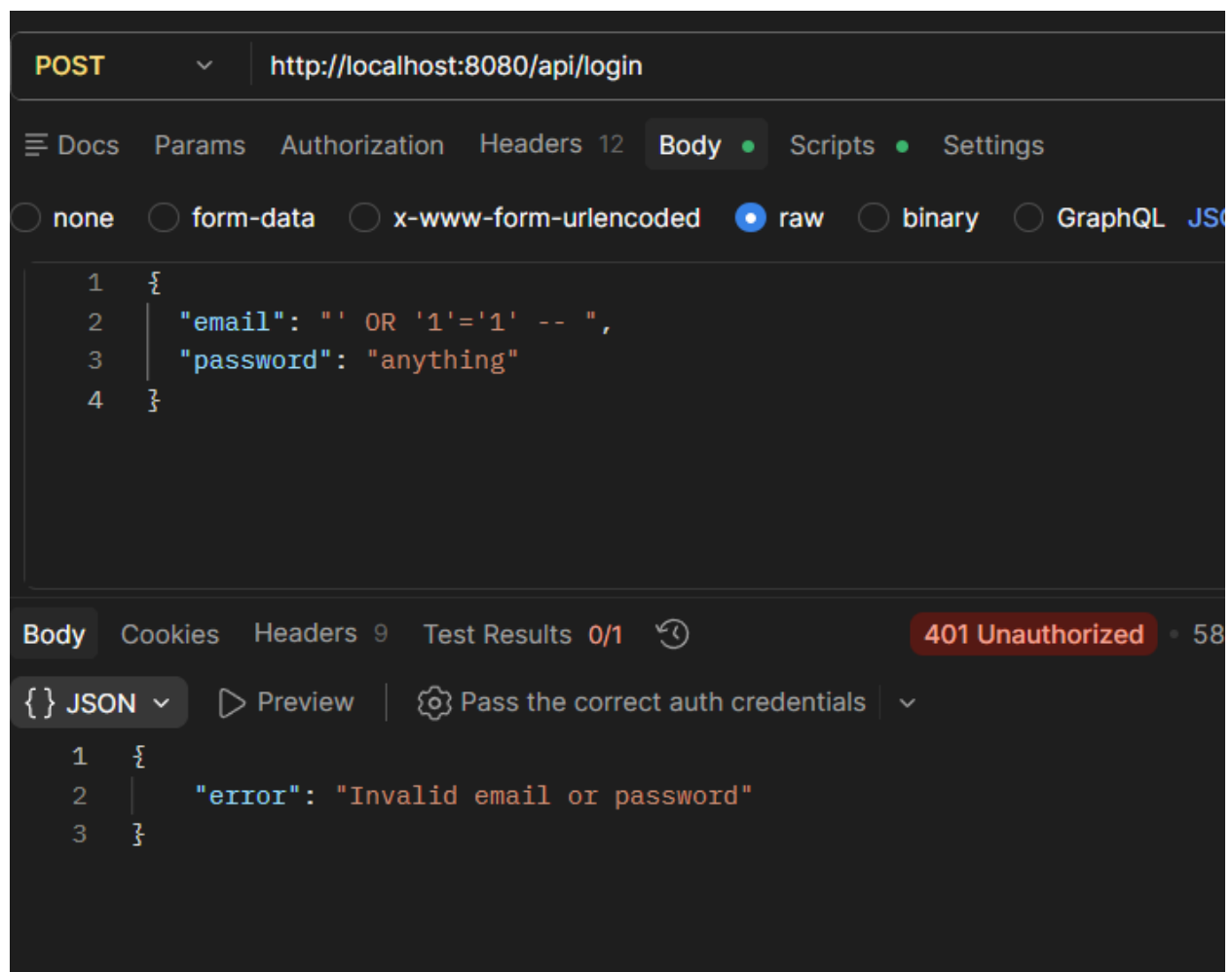
```
1  {
2    "email": "aiman@student.utm.my" -- ",
3    "password": "anything"
4  }
```

Body Cookies Headers 9 Test Results 0/1 🕒

401 Unauthorized • 59 ms •

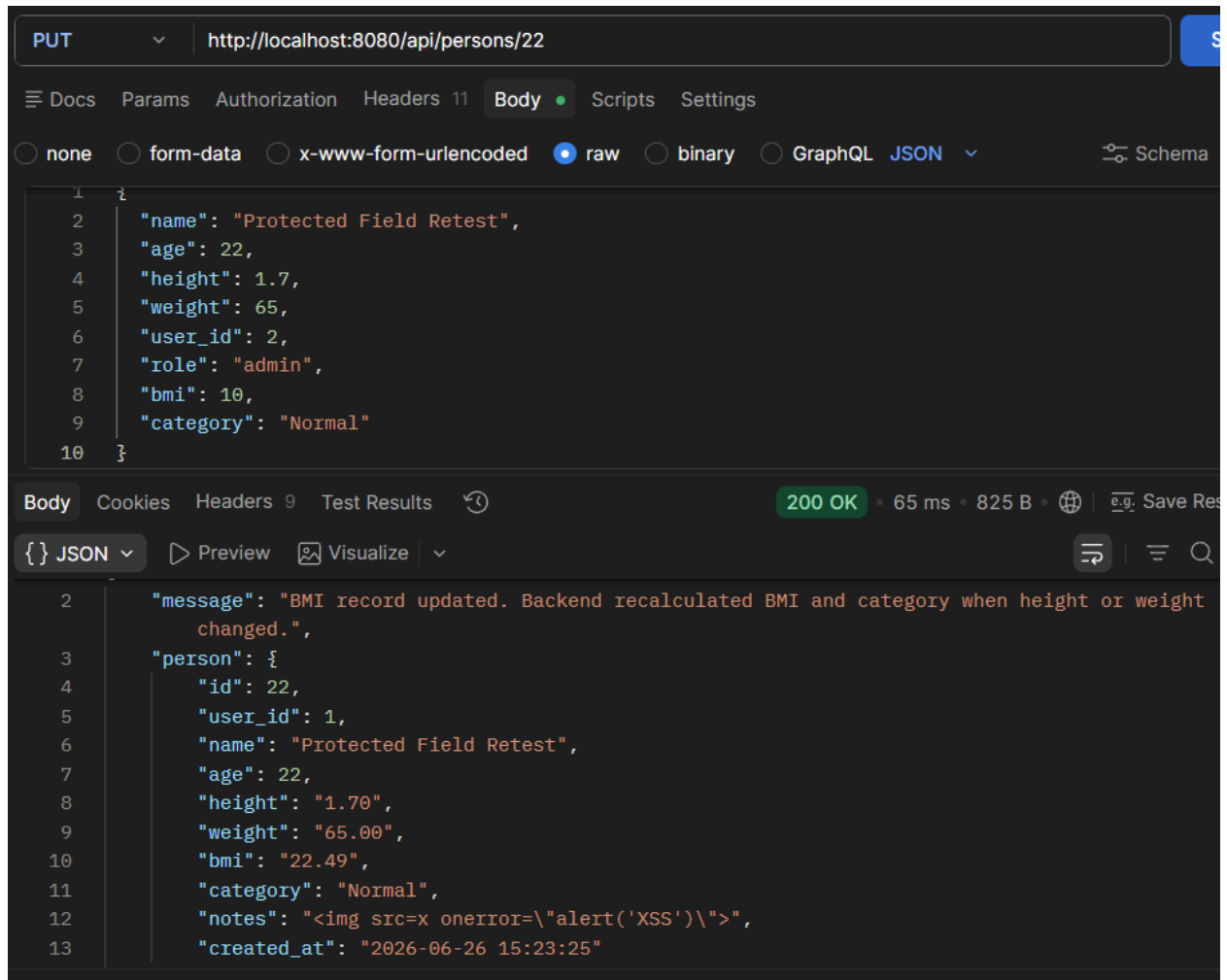
{ } JSON ▼ ▶ Preview | ⚙️ Pass the correct auth credentials | ▼

```
1  {
2    "error": "Invalid email or password"
3  }
```



Result: SQL Injection input was treated as normal data.

Test 8: Try to Update Protected Fields



Result: Protected fields were rejected or ignored.

5.0 Reflections

a) Which weakness was easiest to find? Why?

- The easiest weakness to find was invalid BMI data validation. Because the invalid data could be tested directly using Postman by sending empty name, negative age, zero height, and much more. Therefore, the result was easy to observe because the backend accepted the request instead of rejecting it.

b) Which weakness was most dangerous? Why?

- The most dangerous weakness was broken access control, especially when a normal user could access another user's BMI record or staff and admin routes. This is dangerous because it can expose private user data and allow unauthorized users to perform actions that should only be available to staff or admin like delete operation.

c) Why is frontend validation not enough?

- Frontend validation is not enough because it can still be bypassed using tools such as Postman, Thunder Client, or browser DevTools. So, a user can send requests directly to the backend without following the frontend rules. Therefore, important validation must also be enforced at the backend.

d) Why should BMI calculation be performed at the backend?

- BMI calculation should be performed at the backend because users can manipulate values sent from the frontend. If BMI and category are calculated only at the frontend, users may change the BMI value or category before sending the request. The backend should control this logic to make sure the stored BMI result is correct and trusted.

e) What is the difference between authentication and authorization?

- Authentication means checking who the user is, such as verifying login using email and password. Authorization means checking what the user is allowed to do after login. For example, a user may be logged in, but that does not mean the user can access admin functions.

f) What is the difference between RBAC and owner-based access control?

- Role-Based Access Control can control access based on user roles such as user, staff, or admin. Owner-based access control checks whether the requested data belongs to the logged-in user. For example, a normal user can manage their own BMI records, but cannot access another user's BMI records.

g) Why is JWT alone not enough to protect all data?

- JWT only proves the identity of a logged-in user after the token is verified. It does not automatically mean the user is allowed to access every route or every record. The backend still needs to check the user's role and ownership before allowing access to protected data.

h) Why should API responses exclude password_hash?

- API responses should exclude password_hash because it is sensitive information. The frontend does not need password_hash to display user data. If password_hash is exposed, it may increase the risk of credential attacks if the response is leaked or viewed by unauthorized users.

i) Why is Vue route guard not enough to protect backend routes?

- Vue route guard only controls what the user can see or access in the frontend. It does not protect the backend API directly. A user can still call the backend route using Postman or Thunder Client. Therefore, backend routes must also verify JWT, role, and ownership.

j) How did you prove that your security fix works?

- The security fixes were proven by retesting the same cases before and after the fixes. Before the fixes, the system accepted invalid data, allowed unauthorized access, exposed sensitive data, and allowed unsafe input. After the fixes, the backend returned proper error responses such as 400 Bad Request or 403 Access Denied, sensitive fields were removed, and XSS payloads were displayed as text instead of executing. Screenshots and API responses were included as evidence (Refer 4.0 Phase 4).